

DISTRIBUTED SYSTEMS BCE-C611

Exam-Ready Complete Answer Guide

Subject	Distributed Systems
Code	BCE-C611
Exam Type	Sessional + Semester
Max Marks	20 + 30 + 40 + 40
Coverage	All Questions — All Sections

Covers all questions from First Sessional, Semester Exam (Short + Medium + Long Answer sections)

SECTION 1 — FIRST SESSIONAL EXAMINATION (Max Marks: 20, Time: 1 Hr)

Q1. Write a short note on Distributed Computing and its characteristics.

[5 marks]

Definition

Distributed computing is a model in which multiple autonomous computers communicate through a network to achieve a common goal, appearing as a single coherent system to the end user.

Key Characteristics

- **Concurrency:** Multiple components execute simultaneously and share resources.
- **No global clock:** Nodes have independent clocks; synchronization is a fundamental challenge.
- **Independent failures:** Components can fail independently without halting the entire system.
- **Resource sharing:** Hardware, software, and data are shared across the network.
- **Scalability:** System capacity can be increased by adding more nodes.
- **Transparency:** Hides complexity — access, location, replication, and migration transparency.
- **Heterogeneity:** Nodes may differ in hardware, OS, and programming languages.
- **Openness:** Standard protocols and interfaces allow interoperability.

■ Always mention 'no global clock' and 'independent failures' — these are the key distinguishing features vs parallel systems.

Q2. What are the different challenges in distributed systems?

[5 marks]

Major Challenges

- **Heterogeneity:** Different hardware, OSes, and protocols must interoperate. Solved using middleware.
- **Openness:** System should be extensible — new components must integrate without disruption.
- **Security:** Data must be protected during transmission. Threats include eavesdropping, replay attacks, and impersonation.
- **Scalability:** System must perform well as users and nodes increase. Bottlenecks must be avoided.
- **Failure handling:** Node or network failures occur independently. System must detect and recover gracefully.
- **Concurrency:** Shared resources accessed simultaneously may cause inconsistency if not managed.
- **Transparency:** Access, location, migration, and replication transparency must be maintained.
- **Quality of Service (QoS):** Meeting reliability, timing, and performance guarantees.

■ Memory trick: H-O-S-S-F-C-T-Q — Heterogeneity, Openness, Security, Scalability, Failure, Concurrency, Transparency, QoS.

Q3. Explain client-server communication and RPC with diagram.

[5 marks]

Client-Server Communication

In this model, a client sends a request to the server over a network. The server processes it and sends back a response. Communication can be synchronous (blocking) or asynchronous (non-blocking).

Remote Procedure Call (RPC)

RPC allows a program to execute a procedure on a remote machine as if it were local. The mechanism hides all network complexity from the programmer.

Steps in RPC

Step	Action
1	Client calls a local stub procedure (client stub).
2	Client stub marshals (packs) parameters into a network message.
3	Message is sent to the server over the network.
4	Server stub unmarshals the message and calls the actual server procedure.
5	Server executes the procedure and returns the result to the server stub.
6	Result is marshalled and sent back to the client stub, which returns it to the caller.

RPC Architecture Diagram (Text)

Client → [Client Stub] → [Network Transport] → [Server Stub] → Server Procedure

Server Procedure → [Server Stub] → [Network Transport] → [Client Stub] → Client

Key Terms

- **Marshalling:** Packing procedure arguments into a transmittable byte format.
- **Stub:** Auto-generated proxy code that hides network complexity.
- **IDL (Interface Definition Language):** Defines the interface between client and server.

Q4. What are the different types of distributed systems and their advantages/disadvantages?

[10 marks]

Type	Description	Advantages	Disadvantages
Distributed Computing	Cluster & Grid for HPC tasks	High performance, parallel processing	Complex mgmt, high cost
Distributed Information	Databases, ERP, transaction systems	Centralized access, consistency	Bottlenecks, SPOF
Pervasive / IoT	Mobile, sensor networks, smart devices	Ubiquitous access, real-time data	Security risks, unstable connections
Peer-to-Peer	All nodes equal (BitTorrent, Gnutella)	No central failure, scalable	Hard to manage, security issues
Client-Server	Dedicated servers serve clients	Centralized control, easy maintenance	Server bottleneck, SPOF

Overall Advantages of Distributed Systems

- Resource sharing and economies of scale across nodes.
- Fault tolerance through redundancy — one node's failure does not stop the system.
- Geographical distribution — services accessible worldwide.
- Incremental scalability — capacity added without redesign.

Overall Disadvantages

- Complex software design and harder testing/debugging.
- Network dependency — latency and failures impact performance.

- Security vulnerabilities due to open network communication.
- Consistency and synchronization challenges across nodes.

SECTION 2 — SHORT ANSWER QUESTIONS (5 × 6 = 30 Marks, ~150 words each)

Q1. List the limitations of Distributed Systems.

[6 marks]

- **Complexity:** Designing, implementing, and managing distributed systems is far more complex than centralized ones.
- **Network dependency:** Performance depends heavily on network reliability and available bandwidth.
- **Security threats:** Data transmitted over networks is vulnerable to interception, spoofing, and attacks.
- **Consistency issues:** Maintaining consistent data across replicated nodes is difficult (CAP theorem trade-offs).
- **Difficult debugging:** Errors may be non-deterministic, timing-dependent, and hard to reproduce.
- **Lack of global clock:** Coordinating time across nodes is inherently unreliable.
- **Partial failures:** A component may fail while others continue, leading to unpredictable states.
- **Higher latency:** Network communication overhead increases response time compared to local procedure calls.

Q2. Explain Sun NFS (Network File System).

[6 marks]

Sun NFS (Network File System) is a distributed file system protocol developed by Sun Microsystems that allows a client to access files over a network as if they were on local storage.

Architecture

- **NFS Server:** Exports file systems to be accessed remotely by clients.
- **NFS Client:** Mounts remote file systems using the NFS protocol.
- **VFS (Virtual File System):** Provides a uniform interface; NFS sits below it.
- **RPC-based:** Uses Remote Procedure Calls for client-server communication.

Key Features

- Stateless server design — each request is self-contained, improving fault tolerance.
- Transparent access — users cannot tell if a file is local or remote.
- File locking via NLM (Network Lock Manager).
- Uses UDP or TCP as the transport protocol.

■ *NFS Versions: NFSv2, NFSv3, NFSv4. NFSv4 is stateful and supports strong security via Kerberos.*

Q3. Explain Terms: Event, Process, Threads, Operating System, Client-Server.

[6 marks]

- **Event:** A discrete action or occurrence in a distributed system (e.g., sending a message, executing an instruction). Events establish causal ordering via logical clocks (Lamport timestamps).
- **Process:** An instance of a running program with its own address space. In distributed systems, multiple processes run on different nodes and communicate via message passing.
- **Thread:** A lightweight unit of execution within a process. Threads share the same memory space and are used to handle multiple concurrent requests efficiently (e.g., a multi-threaded server).

- **Operating System:** Software managing hardware resources. In distributed systems, the OS (and middleware layer) provides abstractions like IPC, file access, and remote invocation.
 - **Client-Server:** Architectural model where the client requests services and the server provides them. The server is always listening; the client initiates communication. Example: web browser (client) and web server (server).
-

Q4. How is client-server communication possible in distributed systems?

[6 marks]

Client-server communication in distributed systems is achieved through various mechanisms:

- **Sockets:** Low-level API for TCP/UDP-based communication. Client opens a socket to the server's IP and port; data is exchanged as byte streams.
- **RPC (Remote Procedure Call):** Client calls a procedure that runs on the server. Stubs handle marshalling/unmarshalling transparently.
- **HTTP/REST:** Web-based communication using HTTP methods (GET, POST, PUT, DELETE). Stateless, widely used.
- **Message Queues:** Asynchronous communication using queues (RabbitMQ, Kafka). Client sends a message to the queue; server reads from it independently.
- **RMI (Remote Method Invocation):** Java-based mechanism to invoke methods on remote objects.

Communication Modes

- **Synchronous:** Client blocks and waits for the server's response before continuing.
 - **Asynchronous:** Client continues execution; response handled via callbacks, futures, or polling.
-

Q5. Discuss the different APIs used in the Internet.

[6 marks]

- **Socket API:** POSIX standard — `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `send()`, `recv()`. Foundation of all network communication.
 - **REST API:** Uses HTTP. Stateless, resource-based. Most widely used for web services. Methods: GET, POST, PUT, DELETE.
 - **SOAP API:** XML-based protocol. Uses WSDL to describe services. Strict, formal — common in enterprise systems.
 - **GraphQL:** Query language for APIs allowing clients to request exactly the fields they need — avoids over-fetching.
 - **gRPC:** Google's RPC framework using Protocol Buffers. High performance, supports streaming, ideal for microservices.
 - **WebSocket API:** Full-duplex communication over a single TCP connection — used for real-time apps (chat, gaming, live feeds).
 - **DNS API:** Used to resolve domain names to IP addresses programmatically.
-

Q6. Explain the synchronization with physical and logical clocks.

[6 marks]

Physical Clocks

Hardware-based clocks (oscillators) that drift over time (up to ~1 second/day). Synchronization protocols:

- **NTP (Network Time Protocol):** Synchronizes clocks over the internet to within milliseconds of UTC using a hierarchy of servers.

- **Cristian's Algorithm:** Client asks time server, adjusts for half the round-trip delay.
- **Berkeley Algorithm:** Master polls all slaves, computes average, sends correction to each.

Logical Clocks

Capture causal ordering of events, not actual wall-clock time.

- **Lamport Timestamps:** Each process has counter C . Internal event: $C++$. Send: $C++$, attach. Receive: $C = \max(C, \text{received}) + 1$. If $a \rightarrow b$ then $L(a) < L(b)$, but converse not guaranteed.
- **Vector Clocks:** Each process maintains vector of size n . Captures causality precisely — $V(a) < V(b)$ iff a causally precedes b .

■ *Logical clocks don't tell real time — they define what happened before what. Vector clocks are strictly stronger than Lamport timestamps.*

Q7. Explain domain names and web servers with examples.

[6 marks]

Domain Names

A domain name is a human-readable address for a web resource (e.g., `www.google.com`). DNS (Domain Name System) translates it to an IP address.

- **Structure:** `host.subdomain.domain.TLD` — e.g., `mail.cs.university.edu`
- **DNS Resolution:** Recursive query — Root Server → TLD Server → Authoritative Nameserver → IP returned.
- **DNS Records:** A (IPv4 address), AAAA (IPv6), CNAME (alias), MX (mail), NS (nameserver).

Web Servers

A web server handles HTTP/HTTPS requests and delivers content to clients.

- **Apache HTTP Server:** Open-source, highly configurable, module-based.
- **Nginx:** High performance, event-driven, excellent for static content and reverse proxy.
- **IIS (Internet Information Services):** Microsoft's web server for Windows environments.
- **Node.js (Express):** JavaScript-based server for dynamic applications.

■ *Example flow: User types `www.example.com` → DNS resolves to `93.184.216.34` → Browser sends HTTP GET to that IP → Web server responds with HTML.*

Q8. Explain external data representation and marshalling concepts in detail.

[6 marks]

External Data Representation (XDR)

XDR is a standard for data serialization — converting data into a machine-independent format for transmission across heterogeneous systems. Developed by Sun Microsystems for NFS/RPC.

- Data encoded in big-endian 32-bit format. Supports integers, floats, strings, arrays, structures.
- Alternatives: ASN.1 (network protocols), JSON, XML, Google Protocol Buffers (Protobuf).

Marshalling

Marshalling converts in-memory objects into a transmittable byte sequence. Unmarshalling is the reverse.

- **Why needed?** Different machines have different byte orders (big-endian vs little-endian), pointer sizes, and data alignments.
- **Steps:** Flatten object → convert to canonical format → transmit → reconstruct at destination.
- **Tools:** Java Serialization, Python Pickle, Google Protobuf, Java RMI marshalling.

■ *Analogy: Marshalling = 'packing a suitcase' for the journey, unmarshalling = 'unpacking' at the destination.*

Q9. Explain Process States in detail with diagram.

[6 marks]

Process States

State	Description
New	Process is being created; resources are being allocated.
Ready	Process is loaded in memory, waiting for CPU allocation.
Running	Process is currently executing on the CPU.
Blocked / Waiting	Process is waiting for I/O completion or a resource.
Terminated	Process has finished execution; resources being freed.
Suspended Ready	Swapped out to disk but would run if loaded back.
Suspended Blocked	Swapped out to disk AND waiting for an event.

State Transitions (Diagram)

New → Ready (admitted)

Ready → Running (CPU scheduled)

Running → Ready (preempted by scheduler)

Running → Blocked (I/O or event wait)

Blocked → Ready (I/O complete / event occurred)

Running → Terminated (process exits)

In Distributed Systems

- Remote processes may block waiting for network replies.
- Process migration can move a process between nodes mid-execution.
- Global state snapshots (Chandy-Lamport algorithm) are needed for distributed debugging.

Q10. Explain the various components of RMI in Java with suitable example.

[6 marks]

Java RMI (Remote Method Invocation) allows a Java object on one JVM to invoke methods on a Java object in another JVM, possibly on a different machine.

Component	Role
Remote Interface	Defines methods to be exposed. Extends <code>java.rmi.Remote</code> .
Remote Object (Server)	Implements the remote interface. Extends <code>UnicastRemoteObject</code> .
Stub (Client-side proxy)	Represents remote object on client. Marshals method calls.
Skeleton (Server-side)	Unmarshals requests, calls actual method (older JDK).
RMI Registry	Naming service (<code>rmiregistry</code>) — servers register, clients look up.
Remote Reference Layer	Manages lifecycle and references to remote objects.
Transport Layer	TCP-based transport for all communication.

■ *Example flow: Server: Naming.rebind('Hello', obj) → Client: Naming.lookup('rmi://server/Hello') → client.sayHello() → stub marshals → network → skeleton unmarshals → server executes → result returned.*

SECTION 3 — MEDIUM ANSWER QUESTIONS (4 × 10 = 40 Marks)

Q1. Explain the different consistency models for distributed systems.

[10 marks]

When multiple nodes replicate data, consistency models define which values a read operation may return.

Model	Description	Example
Strict Consistency	Any read returns the most recent write. Requires global time.	Theoretical, impossible in practice.
Linearizability	Operations appear instantaneous at some point between request and finish. Strongest practical model.	Google Spanner
Sequential Consistency	All processes see operations in the same order, not necessarily real time.	GPSS, many VMs
Causal Consistency	Causally related writes seen in same order by all. Causally unrelated writes may differ.	Scalable, more flexible
Eventual Consistency	If no new updates, all replicas eventually converge.	Weak, but DynamoDB, Redis
Release Consistency	Updates propagated when a synchronization lock is released.	Distributed shared memory
Entry Consistency	Data accessed via synchronization variables (locks) is consistent at lock acquisition.	IBM system z

■ **CAP Theorem:** A distributed system can guarantee only 2 of 3: Consistency, Availability, Partition tolerance. Choose your consistency model based on this trade-off.

Q2. Describe the various RPC protocols supporting client-server communication.

[10 marks]

RPC Protocol Operation Steps

1. Client calls client stub.
2. Stub marshals parameters, sends message to server.
3. Server OS receives message, passes to server stub.
4. Stub unmarshals, calls server procedure.
5. Server executes, returns result to stub.
6. Result marshalled, sent back, client stub returns to caller.

RPC Binding

- **Static binding:** Server address hardcoded at compile time.
- **Dynamic binding:** Server registers with a binder (portmapper/registry); client queries at runtime.

RPC Failure Semantics

- **At-least-once:** Client retries on timeout. Server may execute multiple times. Use only for idempotent operations.
- **At-most-once:** Server tracks request IDs to prevent duplicate execution.
- **Exactly-once:** Ideal but hard to guarantee — requires at-most-once + persistence.

Modern RPC Protocols

Protocol	Description
Sun RPC (ONC RPC)	Used in NFS. Uses XDR for encoding. Portmapper for binding.
DCE RPC	Distributed Computing Environment RPC. UUID-based interfaces.

gRPC	Google's modern RPC over HTTP/2 using Protocol Buffers. Supports streaming.
XML-RPC	Uses XML to encode calls; HTTP as transport. Predecessor to SOAP.
JSON-RPC	Lightweight RPC using JSON. Simple, human-readable.

Q3. Describe CORBA RMI and its services.

[10 marks]

CORBA (Common Object Request Broker Architecture) is a middleware standard defined by OMG enabling language- and platform-independent distributed object communication.

Component	Role
ORB (Object Request Broker)	Core middleware — routes requests between clients and distributed objects.
IDL (Interface Definition Language)	Language-neutral interface definition. IDL compilers generate stubs/skeletons.
IIOP	Internet Inter-ORB Protocol — standard for ORB-to-ORB communication over TCP/IP.
Stub / Skeleton	Client stub marshals calls; Server skeleton unmarshals and invokes implementation.
POA (Portable Object Adapter)	Maps object references (IORs) to actual implementations.
IOR (Interoperable Object Reference)	Unique reference containing host, port, and object key.

CORBA Services (CORBAServices)

- Naming Service — maps names to object references.
- Trading Service — service discovery based on properties.
- Event Service — asynchronous communication via event channels.
- Transaction Service — distributed ACID transactions.
- Security Service — authentication, authorization, encryption.
- Persistence Service — storing object state to persistent storage.
- Lifecycle Service — creating, copying, moving, and deleting objects.
- Concurrency Control Service — managing concurrent access.

■ *CORBA vs RMI: CORBA is language-independent (Java, C++, Python etc.) while Java RMI is Java-only. CORBA is more complex but more interoperable.*

Q4. Differentiate TCP stream communication and Client-Server Communication.

[10 marks]

Aspect	TCP Stream Communication	Client-Server Communication
Level	Low-level transport layer	Application-level architectural model
Protocol	Uses TCP (Transmission Control Protocol)	Uses TCP, UDP, HTTP, RPC, etc.
Connection	Byte-stream; connection-oriented	Request-response or message-based
Ordering	Guarantees in-order byte delivery	Ordering depends on protocol used
Reliability	Built-in: ACK, retransmission, error correction	Depends on underlying transport protocol
Abstraction	Raw bytes; no message boundaries defined	Messages have defined structure/format
State	Stateful connection (3-way handshake)	Can be stateful or stateless (e.g., HTTP)
Example	Socket programming API	HTTP server, REST API, RPC server

TCP Communication Steps

- Server: socket() → bind() → listen() → accept()
- Client: socket() → connect()
- Data exchange: send() / recv()
- Termination: close()

Q5. What is name service? Goals? How implemented? What is directory service?

[10 marks]

Name Service

A name service translates human-readable names to network addresses or object references. It resolves 'what' (name) to 'where' (address or reference).

Goals

- Location transparency — access resources by name, not physical location.
- Scalability — must handle millions of names efficiently (e.g., DNS).
- Availability — must be highly available; failure cripples the entire system.
- Consistency — name-to-address bindings must be current and accurate.
- Security — prevent unauthorized modification of bindings.

Implementation

- **DNS:** Hierarchical and distributed. Root → TLD → Authoritative nameservers. Caching reduces query load.
- **CORBA Naming Service:** Maps object names to IORs (Interoperable Object References).
- **JNDI (Java Naming and Directory Interface):** Java API for accessing multiple naming/directory services.
- **LDAP:** Lightweight Directory Access Protocol — combines naming and directory functions.

Directory Service

A directory service is an extended name service that stores attributes of resources (name, address, capabilities, properties). Users can search by any attribute, not just name.

- **Examples:** Active Directory (Microsoft), OpenLDAP, X.500.
- **Key Difference:** Name service = name→address. Directory service = search by any attribute.

Q7. Compare and contrast DNS and GNS with their advantages and disadvantages.

[10 marks]

Aspect	DNS (Domain Name System)	GNS (Global Name Service)
Scope	Internet-wide name resolution	Global, for wide-area distributed systems
Structure	Hierarchical tree (Root→TLD→Domain)	Cell-based hierarchy
Design goal	Name-to-IP for internet hosts	Flexible naming for distributed objects
Consistency	Eventual (TTL-based caching)	Stronger consistency model
Mobility support	Limited; requires manual DNS update	Better support for dynamic addresses
Security	DNSSEC (optional extension)	Built-in security design

Scalability	Highly scalable — distributed + cached	Designed for global scalability
-------------	--	---------------------------------

DNS — Advantages & Disadvantages

- **Advantages:** Universal adoption, very fast due to caching, fully decentralized.
- **Disadvantages:** Eventual consistency (stale cache), vulnerable to DNS spoofing, no native object mobility.

GNS — Advantages & Disadvantages

- **Advantages:** Better consistency guarantees, supports dynamic migration, built-in security.
- **Disadvantages:** Not universally adopted, more complex to implement and deploy.

Q8. Explain: a) Global State b) Mutual Exclusion c) Logical Exclusion

[10 marks]

a) Global State

The global state of a distributed system is the collection of all local process states plus the state of all communication channels (messages in transit). Used for: distributed snapshots, deadlock detection, and garbage collection.

Chandy-Lamport Algorithm: Initiator saves local state and sends a MARKER on all outgoing channels. On receiving a MARKER for the first time, a process saves its state and starts recording channel state. Produces a consistent cut.

b) Mutual Exclusion in Distributed Systems

Ensures only one process accesses a critical section at a time. No shared memory — must use message passing.

Algorithm	Approach	Messages/Entry	Weakness
Centralized	Coordinator grants access on REQUEST	2 (Request+grant+release)	Single point of failure
Ricart-Agrawala	Broadcast REQUEST; wait for n-1 REPLYs	2(n-1)	No fault tolerance
Token Ring	Token circulates; only holder enters CS	avg (0 to n-1 hops)	Token loss = system failure
Maekawa Voting	Need votes from quorum ($\sqrt{n}$ processes)	$2\sqrt{n}$	Deadlock possible

c) Logical Exclusion

Uses Lamport timestamps to order requests for the critical section. A process enters the CS only when it holds the request with the smallest timestamp among all known pending requests. Ensures consistent global ordering without physical clocks.

SECTION 4 — LONG ANSWER QUESTIONS (4 × 10 = 40 Marks)

Q1. Explain the architectural models of distributed systems, highlighting advantages and disadvantages of each.

[10 marks]

Model	Description	Advantages	Disadvantages
Client-Server	Clients request; dedicated servers respond.	Simple, centralized control, easy maintenance.	Server bottleneck, single point of failure.
Peer-to-Peer (P2P)	All nodes are equal — act as both clients and servers.	No SPOF, decentralized (BitTorrent).	Hard to manage, inconsistent performance, security issues.
Layered Architecture	System divided into layers (App → Middleware → OS → HW). Each layer provides services to layers above.	Modularity, easier to debug.	Added complexity, debugging complex.
Object-Based	Distributed objects communicate via remote calls (RMI, CORBA, SOAP).	Reusable, platform-independent.	Performance overhead, tight coupling.
Data-Centered	Components interact through a shared central repository (pages, objects).	Simple, easy to implement.	Central bottleneck, consistency challenges.
Event-Based (Pub-Sub)	Publishers emit events; subscribers receive matching messages.	Loose coupling, highly scalable.	Debugging complex, harder to debug.
Microservices	Application split into small, independent deployable units.	Independent scaling, deployment, updates.	High overhead, operational complexity, distributed transactions.

■ *Exam tip: For each model, always mention 1 real-world example (e.g., P2P→BitTorrent, Event-based→Kafka, Microservices→Netflix).*

Q2. Discuss design issues of DSM (Distributed Shared Memory) in detail.

[10 marks]

DSM provides an abstraction of shared memory to processes on different machines. Processes read/write a common address space maintained by middleware, even though physical memory is distributed.

Key Design Issues

- **Granularity:** Unit of sharing: page-based (pages), object-based (objects), or variable-based. Fine granularity reduces false sharing but increases overhead.
- **Consistency Model:** Which model? Sequential, causal, release, entry? Weaker models offer better performance.
- **Replication:** Keeping copies at multiple nodes improves read performance but complicates update propagation.
- **False Sharing:** Two processes use different variables in the same page; one write invalidates the other's copy unnecessarily. Solved by finer granularity.
- **Synchronization:** Must provide synchronization primitives (locks, barriers) to coordinate access to shared data.
- **Fault Tolerance:** Node crashes must not corrupt shared state. Requires checkpointing or replication strategies.
- **Scalability:** Centralized directories bottleneck. Distributed directories (e.g., Ivy protocol) needed for large systems.
- **Thrashing:** Pages ping-pong between nodes if two nodes repeatedly compete for the same page.

Update Protocols

- **Invalidation protocol:** On write, all other copies are invalidated. Next read causes a fetch.
- **Update protocol:** On write, all copies are updated immediately. Better for read-heavy sharing.
- **Release consistency:** Updates propagated on lock release — reduces synchronization frequency.

Q3. Discuss the authentication framework in Distributed Systems.

[10 marks]

In distributed systems, communicating parties cannot physically verify each other. Authentication proves identity before granting access.

Authentication Approaches

- **Password-based:** Simplest form. Must be combined with encryption (TLS) to prevent eavesdropping.
- **Public Key Authentication:** Server sends a challenge; client signs with private key; server verifies with public key.
- **Certificate-based (PKI):** Digital certificates from trusted Certificate Authorities (CAs) bind identities to public keys. Used in SSL/TLS.
- **Kerberos:** Trusted third-party authentication using tickets and session keys. Never sends passwords over the network.

Kerberos Protocol (5 Steps)

Step	Action
1	Client requests Ticket Granting Ticket (TGT) from Authentication Server (AS).
2	AS returns TGT encrypted with client's password-derived key.
3	Client presents TGT to Ticket Granting Server (TGS) and requests service ticket.
4	Client presents service ticket to the actual application server.
5	Server authenticates using session key embedded in the ticket.

SSL/TLS Handshake (Simplified)

- Client Hello — lists supported cipher suites.
- Server Hello + Certificate (signed by CA).
- Client verifies certificate with trusted CA.
- Session keys exchanged via asymmetric crypto.
- Secure symmetric-encrypted channel established.

■ *Key distinction: Authentication = who are you? | Authorization = what can you do? | Non-repudiation = proof you did it.*

Q4. IPC in UNIX — describe types of interprocess communication mechanisms, their strengths and weaknesses.

[10 marks]

Mechanism	Description	Strength	Weakness
Pipes	Unidirectional byte stream. Anonymous, built-in namespace. Child-only pipe (parent processes only).	Simple, built-in namespace.	Child-only pipe (parent processes only).
Named Pipes (FIFO)	Like pipes but with filesystem name. Unidirectional.	Unidirectional.	Can still be unidirectional.
Signals	Asynchronous notifications (SIGKILL, SIGTERM, etc.). No data transfer.	Simple, fast.	No data transfer, unreliable ordering.
Message Queues	Kernel-maintained linked list of messages. Supports persistent storage.	Structured, persistent storage.	Kernel overhead, size limits.
Shared Memory	Multiple processes map same physical memory.	Highest speed, fastest IPC.	Requires external synchronization.
Semaphores	Integer counters: P() wait, V() signal. Prevents race conditions.	Prevents race conditions.	Risk of deadlock if misused.
Sockets	Communication endpoint. Supports TCP, UDP, Unix domain sockets.	Secure, UDP, Unix domain sockets.	Higher overhead than shared mem.

Memory-mapped Files	File mapped via mmap(). Changes visible to all processes mapping it.	File shared between processes mapping it.	File system dependency
---------------------	--	---	------------------------

■ *Speed order (fastest → slowest): Shared Memory > Memory-mapped files > Message queues > Pipes > Sockets*

Q5. Describe OS mechanisms for distributed objects and remote invocation.

[10 marks]

The OS layer provides fundamental abstractions that enable distributed objects and remote invocations.

Key OS Mechanisms

- **Protection:** Memory protection ensures processes cannot corrupt each other. Address spaces isolated — essential for safely running untrusted remote code.
- **Processes and Threads:** OS provides multi-threading to handle concurrent requests. Each RMI call handled by a separate thread. Thread pools limit overhead.
- **Communication (Sockets):** OS provides the socket API as the foundation for network communication. Middleware (CORBA ORB, Java RMI) sits on top of this.
- **OS-level IPC:** For distributed objects on the same machine, shared memory or pipes are faster than network sockets.
- **Scheduling:** OS schedules threads serving remote requests. Priority scheduling can ensure QoS for latency-sensitive remote calls.
- **Object Activation:** OS may load an object from disk when its remote reference is first invoked (e.g., CORBA Portable Object Adapter — POA).

OS Architecture Styles for Distributed Systems

- **Monolithic Kernel:** All OS services in kernel space. Fast but inflexible (Linux, traditional UNIX).
- **Microkernel:** Minimal kernel + user-space servers. Ideal for distributed OSes (e.g., Mach, L4). Easier to customize protocols.
- **Exokernel:** Exposes hardware directly; library OSes provide abstractions. Allows custom distributed protocols with minimal overhead.

Q6. Explain Distributed Mutual Exclusion and its types in detail.

[10 marks]

Distributed mutual exclusion ensures that only one process can access a shared critical section (CS) at a time in a system with no shared memory — only message passing.

Requirements

- **Safety:** At most one process in the CS at any time.
- **Liveness:** Every request to enter the CS is eventually granted.
- **Fairness:** Requests are granted in the order they were made (usually by Lamport timestamp).

Algorithms

Algorithm	Approach	Messages/Entry	Weakness
Centralized	One coordinator grants access. Send REQUEST, receive GRANT, then enter CS.	2n-1	Coordinator is a single point of failure
Ricart-Agrawala	Broadcast REQUEST(ts,pid). Enter CS after receiving n-1 REPLYs.	2n-1	Process failure blocks progress
Lamport's	All processes maintain timestamp-sorted REQUEST queues. Enter CS when no other queues have lower timestamp.	2n-1	High message cost
Token Ring	Token circulates in logical ring. Only token holder may enter CS.	1n	Token loss = deadlock

■ **Quick comparison:** Centralized=simple but SPOF | Ricart-Agrawala=no SPOF but $2(n-1)$ msgs | Token Ring=1 msg avg but token loss dangerous | Maekawa= $\sqrt{n}$ msgs but deadlock risk.

Q7. Discuss clocks, events, and process states in a distributed system. Explain physical and logical clocks, global state, and distributed debugging.

[10 marks]

Events in Distributed Systems

Three types: (1) Internal events — local computation, (2) Send events — transmitting a message, (3) Receive events — receiving a message. Together they form the causal history of a distributed execution.

Physical Clocks and Challenges

Hardware oscillator clocks drift (up to ~1 second/day). Even after synchronization, maximum clock skew ϵ cannot be zero. Cristian's Algorithm and Berkeley Algorithm achieve clock agreement but not perfect sync.

Logical Clocks — Lamport Timestamps

- Each process P_i has counter C_i . On internal event: C_i++ . On send: C_i++ , attach C_i . On receive: $C_i = \max(C_i, \text{received_ts}) + 1$.
- Guarantee: if $a \rightarrow b$ (a happened-before b), then $C(a) < C(b)$. Converse NOT guaranteed.

Vector Clocks

- Each process P_i maintains vector $V[1..n]$. On event at P_i : $V[i]++$. On send: attach V . On receive: $V[j] = \max(V[j], \text{received}[j])$ for all j , then $V[i]++$.
- $V(a) < V(b)$ if and only if a causally precedes b — captures causality precisely, unlike Lamport timestamps.

Global State — Chandy-Lamport Snapshot Algorithm

Step	Action
1	Initiator P_i records its local state. Sends MARKER message on all outgoing channels.
2	Process P_j receiving MARKER for the first time: records its local state, sends MARKER on all its outgoing channels, starts recording channel state.
3	When P_j receives MARKER on channel C_k (already seen MARKER): records channel C_k state = messages received on C_k since last MARKER.
4	Algorithm terminates when all processes have received MARKERS on all incoming channels.

The resulting snapshot is a consistent global state — useful for deadlock detection, distributed debugging, and checkpointing/recovery.

■ **Key insight:** A consistent cut is one where for every message received in the cut, the corresponding send is also in the cut. Chandy-Lamport guarantees this.

Q8. Explain CORBA and its role in distributed systems. Discuss CORBA RMI framework, design, implementation, and services.

[10 marks]

CORBA (Common Object Request Broker Architecture) is a middleware standard by OMG enabling language- and platform-independent distributed object communication. Objects written in Java, C++, Python, etc. can all communicate via CORBA.

Architecture Components

Component	Description
ORB (Object Request Broker)	Core middleware bus — locates remote objects, routes requests, returns results. The central component.
IDL (Interface Definition Language)	Platform-neutral interface definition. IDL compilers generate language-specific stubs and skeletons.
IIOB (Internet Inter-ORB Protocol)	TCP/IP-based protocol for ORB-to-ORB communication. Ensures interoperability across vendors.
Client Stub (Proxy)	Represents remote object on client side. Marshals method calls into IIOB request messages.
Server Skeleton	Unmarshals incoming IIOB requests and dispatches them to the actual object implementation.
POA (Portable Object Adapter)	Manages object lifecycle, maps IORs to implementations, handles activation/deactivation.
IOR (Interoperable Object Reference)	Unique global reference to a CORBA object. Contains host, port, object key, type info.

CORBA RMI — Invocation Flow

- Client obtains IOR of the server object (from Naming Service or IOR string).
- Client calls a method on the local stub (proxy).
- Stub marshals parameters into an IIOB request message.
- Client ORB routes the IIOB message to the server's ORB via TCP/IP.
- Server ORB delivers the message to the server skeleton.
- Skeleton unmarshals the request and calls the actual implementation method.
- Return value is marshalled and sent back as an IIOB reply.
- Client stub receives and unmarshals the reply — caller gets the return value.

CORBA Services Summary

Naming (object lookup), Trading (property-based discovery), Event/Notification (async messaging), Transaction (ACID across objects), Security (auth + crypto), Persistence (state storage), Lifecycle (create/copy/move/delete), Concurrency Control, Time Service.

■ *Real-world CORBA use: Banking systems, Telecom (3GPP), Military C2 systems. Modern replacement: gRPC for new distributed systems.*